

1. Executive Summary

In this project, I developed a machine learning forecasting pipeline to predict hourly yellow taxi demand across NYC taxi zones. I defined demand as the number of yellow taxi pickups in each pickup zone during each hour. This directly supports the assessment objective of forecasting yellow taxi trips or revenue by NYC taxi zone on an hourly basis.

I used 2023 NYC Yellow Taxi trip data as the main dataset and supplemented it with external weather and holiday data. I transformed raw trip-level records into a zone-hour demand table, engineered calendar, weather, holiday, lag, and rolling demand features, compared several regression models, improved the best model through hyperparameter tuning, and created a reusable prediction workflow.

The best model after tuning was a LightGBM Regressor. On the November validation period, the tuned model achieved:

Metric	Value
MAE	2.8905
RMSE	10.2939
WAPE	0.1684
R ²	0.9638

On the final December test period, the model achieved:

Metric	Value
MAE	3.7705
RMSE	13.3534
WAPE	0.2241
R ²	0.9349

The December performance was weaker than the validation performance, which suggests that year-end and holiday-period travel patterns were harder to forecast. However, the model still

captured a large portion of the demand variation and produced usable hourly zone-level forecasts.

2. Problem Understanding

The objective of this assessment was to build a forecasting model that predicts either the number of yellow taxi trips or total revenue for each NYC taxi zone on an hourly basis.

I chose to forecast the number of yellow taxi trips because trip count is a direct measure of passenger demand. For taxi allocation, predicting the number of expected pickups is more immediately useful than predicting revenue. A zone with high expected pickups may need more taxi supply, while a zone with low expected pickups may need fewer taxis.

My final prediction target was:

trip_count = number of yellow taxi pickups in a pickup zone during one hour

Each prediction is therefore defined by:

pickup_hour + PULocationID -> predicted_trip_count

For example, a prediction for PULocationID = 138 at 2023-12-01 08:00:00 represents the expected number of yellow taxi pickups in LaGuardia Airport zone between 8:00 AM and 9:00 AM.

3. Data Sources

3.1 Main Dataset

I used the provided 2023 NYC Yellow Taxi trip records. The raw data consisted of monthly parquet files from January to December 2023. Each row represented one taxi trip and included fields such as pickup time, dropoff time, pickup zone, dropoff zone, trip distance, fare amount, total amount, passenger count, and payment type.

The main columns used during preparation included:

Column	Description
tpep_pickup_datetime	Taxi pickup timestamp
tpep_dropoff_datetime	Taxi dropoff timestamp
PULocationID	Pickup taxi zone ID
DOLocationID	Dropoff taxi zone ID
trip_distance	Trip distance
fare_amount	Fare amount
total_amount	Total charged amount
passenger_count	Recorded passenger count

I also used the taxi zone lookup table to map PULocationID to borough, zone name, and service zone.

3.2 External Weather Data

I supplemented the taxi data with NOAA-style hourly weather observations. I used weather because taxi demand can be affected by rain, visibility, temperature, wind, and other weather conditions.

The weather features I extracted included:

Feature	Description
temp_c	Temperature in Celsius
dew_c	Dew point in Celsius
pressure_hpa	Sea-level pressure
wind_speed_mps	Wind speed in meters per second
visibility_m	Visibility in meters
precip_mm	Hourly precipitation in millimeters
has_precipitation	Indicator for precipitation

I used four NYC-area weather stations:

Station Code	Area
72505394728	NY City Central Park
72503014732	LaGuardia Airport
74486094789	JFK International Airport
72502014734	Newark Liberty International Airport

I assigned Central Park weather to most taxi zones. For airport zones, I used airport-specific weather:

Taxi Zone	Weather Station
Zone 1, Newark Airport	Newark Liberty
Zone 132, JFK Airport	JFK
Zone 138, LaGuardia Airport	LaGuardia

This approach gave airport zones more locally relevant weather conditions.

3.3 Holiday Data

I added US federal holiday information because holidays can change travel patterns, commuting behaviour, nightlife activity, and airport demand.

The holiday features I created were:

Feature	Description
is_holiday	Whether the date is a holiday
holiday_name	Name of the holiday
is_day_before_holiday	Whether the date is one day before a holiday
is_day_after_holiday	Whether the date is one day after a holiday

I included both actual and observed holiday dates. For example, New Year's Day 2023 occurred on January 1, while the observed federal holiday was January 2. I also corrected a detected issue in the holiday file where Memorial Day 2023 appeared incorrectly. I corrected it to May 29, 2023.

4. Data Preparation

The raw taxi data was trip-level, but the forecasting task required hourly zone-level predictions. Therefore, the most important data preparation step was converting individual taxi trips into an hourly demand table.

The transformation was:

one row per trip -> one row per pickup zone per hour

I processed the taxi parquet files month by month to avoid loading the entire year into memory at once.

4.1 Taxi Data Cleaning

I applied the following cleaning rules:

Cleaning Rule	Reason
Kept pickups from 2023 only	Removed invalid timestamps outside the modeling period
Required dropoff time after pickup time	Removed impossible trips
Removed trips longer than 4 hours	reduced extreme trip-duration outliers
Kept pickup and dropoff zones from 1 to 263	Removed unknown and outside-NYC special zones
Removed negative distance values	Negative distances are invalid

Removed negative fare and total amount values	Negative payments likely represent corrections or invalid records
---	---

I did not remove records with `passenger_count = 0` because these can still represent real taxi trips. In this project, the target is trip demand, not passenger volume, so a valid trip record still contributes to demand even if passenger count is missing or incorrectly recorded as zero.

After cleaning, the total number of cleaned trips used for modeling was 37 267 702 trips.

4.2 Hourly Aggregation

I created `pickup_hour` by flooring each pickup timestamp to the nearest hour. Then I grouped records by `pickup_hour` and `PULocationID`.

For each zone-hour, I calculated:

Column	Meaning
<code>trip_count</code>	Number of pickups
<code>revenue</code>	Total revenue
<code>avg_trip_distance</code>	Average trip distance
<code>avg_fare_amount</code>	Average fare amount
<code>avg_trip_duration_min</code>	Average trip duration in minutes
<code>zero_passenger_count</code>	Number of trips with passenger count equal to zero

The modeling target was `trip_count`.

4.3 Complete Zone-Hour Grid

I created a complete grid of all 263 valid taxi zones and all 8760 hours in 2023.

This produced:

263 zones x 8,760 hours = 2303880 rows

This step was important because missing zone-hour combinations should represent zero observed demand, not missing data. If a zone had no pickups during an hour, I filled trip_count with zero.

4.4 Prepared Dataset

The final prepared dataset contained:

Item	Value
rows	2303880
columns	51
Taxi zones	263
Date range	2023-01-01 00:00 to 2023-12-31 23:00
Cleaned trips	37267702
Missing weather rows	0

5. Exploratory Data Analysis

During exploration, I focused on understanding demand patterns over time and across locations. I observed that yellow taxi demand is highly concentrated in Manhattan and major transportation zones such as airports and Midtown.

The highest predicted December demand zones included:

Zone ID	Zone	Borough	Total predicted trips
237	Upper East Side South	Manhattan	190415
236	Upper East Side North	Manhattan	173636
161	Midtown Center	Manhattan	170212

132	JFK Airport	Queens	164386
138	LaGuardia Airport	Queens	121025

These results are reasonable because Manhattan business, residential, entertainment, and airport-related zones naturally have high taxi activity.

I also examined temporal patterns such as hour of day, day of week, weekends, holidays, and month. These patterns are important because taxi demand is not random. It follows daily commuting behaviour, nightlife activity, airport travel patterns, and holiday effects.

6. Feature Engineering

I engineered several groups of features to help the model learn time, location, weather, holiday, and historical demand patterns.

6.1 Calendar Features

I created calendar features from pickup_hour:

Feature	Meaning
hour	Hour of day
dayofweek	Day of week
dayofyear	Day number in year
weekofyear	Week number
month	month
is_weekend	Weekend indicator

These features help the model learn daily, weekly, and seasonal demand patterns.

6.2 Cyclical Time Features

I created cyclical encodings for hour, day of week, and month:

Feature pair	Purpose
hour_sin, hour_cos	Daily cycle
dow_sin, dow_cos	Weekly cycle
month_sin, month_cos	Yearly cycle

I used sine and cosine transformations because time is cyclical. For example, 23:00 and 00:00 are close in real time, but if represented as plain numbers, 23 and 0 appear far apart. Cyclical encoding helps the model understand that these values are close in the time cycle.

6.3 Weather Features

I added weather features because weather can affect taxi demand. Rain, low visibility, high wind, and uncomfortable temperatures can change people's transportation choices.

Weather features included:

temp_c, dew_c, pressure_hpa, wind_speed_mps, visibility_m, precip_mm, and has_precipitation.

6.4 Holiday Features

I added holiday features because holidays can affect commuting, leisure travel, airport demand, and general mobility.

Holiday features included:

is_holiday, holiday_name, is_day_before_holiday, and is_day_after_holiday.

6.5 Lag Features

Lag features were among the most important features. They represent historical demand for the same pickup zone.

I created the below.

Feature	meaning
trip_count_lag_1	Demand 1 hour ago
trip_count_lag_2	Demand 2 hour ago
trip_count_lag_3	Demand 3 hour ago
trip_count_lag_24	Demand yesterday at the same hour
trip_count_lag_48	Demand two days ago at the same hour
trip_count_lag_72	Demand three days ago at the same hour
trip_count_lag_168	Demand one week ago at the same hour

The training data starts on January 8 because trip_count_lag_168 requires 168 previous hours, or one full week of history.

6.6 Rolling Demand Features

I also created rolling average demand features:

Feature	Meaning
trip_count_rolling_24_mean	Average demand over the previous 24 hours
trip_count_rolling_168_mean	Average demand over the previous 168 hours

I shifted demand before calculating rolling averages to avoid data leakage. This ensures the model never uses the current hour's actual demand to predict the current hour.

7. Modeling Approach

This is a regression forecasting problem because the target is a numeric trip count. I did not use classification metrics. I evaluated models using regression and forecasting metrics.

I used a chronological split:

split	period	purpose
train	January to October 2023	Model fitting
Validation	November 2023	Model selection

Test	December 2023	Final evaluation
------	---------------	------------------

I used a time-based split instead of a random split because forecasting models should train on the past and predict the future. A random split would mix future and past records and could lead to unrealistic performance estimates.

8. Model Selection

I compared simple baselines and tree-based regression models.

8.1 Baseline Models

I included baseline models because a machine learning model should improve over simple historical rules.

The baselines were:

Baseline	Meaning
Baseline_lag_24	Predict using yesterday's same hour
Baseline_lag_168	Predict using last week's same hour
Baseline_avg_lag_24_168	Average of yesterday and last week

8.2 Candidate Machine Learning Models

I compared the following models:

model	Reason for inclusion
Decision Tree Regressor	Simple and interpretable
Random Forest Regressor	Ensemble of decision trees, more stable than one tree
XGBoost Regressor	Strong boosted-tree model for tabular data
LightGBM Regressor	Fast boosted-tree model for large tabular

	datasets
CatBoost Regressor	Boosted-tree model that performs well on tabular data

I trained the models on $\log_{10}(\text{trip_count})$ to reduce the influence of extremely high-demand zone-hours. I converted predictions back to trip counts using $\exp(m)$ before evaluation.

8.3 Model Selection Results

The validation results were:

Model	MAE	RMSE	WAPE	R ²
LightGBM Regressor	2.9107	10.3828	0.1696	0.9632
XGBoost Regressor	2.9782	10.8962	0.1735	0.9594
Random Forest Regressor	3.0599	11.4116	0.1783	0.9555
CatBoost Regressor	3.0671	11.0563	0.1787	0.9582
Decision Tree Regressor	3.2387	11.4092	0.1887	0.9555
Average Lag Baseline	3.9965	14.6932	0.2329	0.9262
Last Week Baseline	4.5413	17.8076	0.2646	0.8917
Yesterday Baseline	4.9297	19.1966	0.2873	0.8741

LightGBM performed best on validation WAPE, so I selected it for model improvement.

9. Evaluation Metrics

I used four regression metrics.

9.1 MAE

MAE stands for Mean Absolute Error. It measures the average absolute difference between actual and predicted trips.

A lower MAE is better. If MAE is 2.89, the model is wrong by about 2.89 trips per zone-hour on average.

9.2 RMSE

RMSE stands for Root Mean Squared Error. It penalises large errors more strongly than MAE.

A lower RMSE is better. If RMSE is much larger than MAE, it means the model sometimes makes large errors.

9.3 WAPE

WAPE stands for Weighted Absolute Percentage Error.

It is calculated as:

sum of absolute errors / sum of actual demand

WAPE is useful for demand forecasting because it shows the total forecasting error relative to total demand. A WAPE of 0.1684 means the model's total absolute error was about 16.84% of total actual demand.

9.4 R²

R² measures how much variation in the target is explained by the model.

An R² value closer to 1 is better. An R² of 0.9638 means the model explains about 96.38% of the variation in validation demand.

10. Model Improvement

After selecting LightGBM, I fine-tuned the model using a manual hyperparameter search.

I tuned parameters such as:

Hyperparameter	Purpose
learning_rate	Controls how much each tree contributes
num_leaves	Controls tree complexity

min_child_samples	Helps reduce overfitting
subsample	Uses a row sample for each tree
colsample_bytree	Uses a feature sample for each tree
reg_alpha	L1 regularization
reg_lambda	L2 regularization
n_estimators	Number of boosting rounds

The best tuned validation result was:

Metric	value
MAE	2.8905
RMSE	10.2939
WAPE	0.1684
R ²	0.9638

This slightly improved the validation WAPE from 0.1696 to 0.1684.

The best configuration used:

hyperparameter	value
n_estimators	1000
learning_rate	0.035
num_leaves	160
min_child_samples	80
subsample	0.90
colsample_bytree	0.80
reg_alpha	0.20
reg_lambda	1.00

best_iteration	630
----------------	-----

11. Final Model Evaluation

After tuning, I retrained the final LightGBM model on January through November 2023 and evaluated it on the held-out December 2023 test set.

The December test results were shown below.

Metric	Value
MAE	3.7705
RMSE	13.3534
WAPE	0.2241
R ²	0.9349

The model performed worse on December than on November validation. I believe this is reasonable because December includes year-end travel patterns, holidays, and unusual seasonal behaviour. These factors can make demand harder to forecast.

12. Prediction and Inference

I saved the final trained model using joblib. The saved artifact includes:

Saved item	Purpose
Trained LightGBM model	Used for prediction
Target name	Documents prediction target
Numeric feature names	Rebuilds numeric input features
Categorical feature names	Rebuilds encoded categorical features
Final feature columns	Ensures inference data matches training schema

Best hyperparameters	Documents tuned model settings
Validation metrics	records model-selection performance
Test metrics	Records final held-out performance

The saved model file is:

data/processed/final_lgbm_trip_demand_model.joblib

I also created a reusable inference function called `predict_taxi_demand()`. This function accepts prepared future rows and returns predicted trip counts.

Inference data means new data that is passed into the trained model for prediction. In this project, inference data must contain the same kind of prepared features used during training, such as pickup hour, pickup zone, weather features, holiday features, lag features, and rolling demand features.

I generated December hourly predictions and saved them as:

data/processed/december_lgbm_predictions.parquet

I also aggregated December predictions by zone and saved them as:

data/processed/december_lgbm_predictions_by_zone.csv

13. Key Findings

The model found strong demand patterns across time and location. The highest predicted demand zones were mostly in Manhattan and airport zones, which aligns with expectations for yellow taxi activity.

The top predicted December zones were:

Zone ID	Zone	Borough	Total predicted trips
237	Upper East Side South	Manhattan	190415
236	Upper East Side North	Manhattan	173636
161	Midtown Center	Manhattan	170212

132	JFK Airport	Queens	164386
162	Midtown East	Manhattan	141125
142	Lincoln Square East	Manhattan	136145

These findings are consistent with real-world expectations. Dense Manhattan zones and airports are major sources of yellow taxi demand.

The model also showed that lag features were important. This means recent historical demand and same-hour demand from previous days or weeks are strong predictors of future taxi pickups.

14. Assumptions

I made the following assumptions:

1. I defined taxi demand as the number of yellow taxi pickups, not total revenue.
2. I used PULocationID because taxi allocation decisions depend on where passengers need taxis.
3. I treated missing zone-hour combinations as zero observed demand.
4. I removed zones 264 and 265 because they represent unknown or outside-NYC locations.
5. I removed invalid and extreme trip records to improve data quality.
6. I used Central Park weather for most zones and airport-specific weather for airport zones.
7. I assumed holiday effects can influence not only the holiday itself, but also the day before and after the holiday.
8. I used time-based train, validation, and test splits to simulate real forecasting conditions.
9. I assumed future inference data would have available weather, holiday, calendar, and lag features.

15. Limitations

There are several limitations to this project.

First, I used only one year of taxi data. More years of data would help the model learn stronger seasonal patterns and unusual holiday effects.

Second, the model predicts trip count, not taxi supply. High demand does not always mean unmet demand. To fully optimize taxi allocation, taxi availability and driver behaviour would also be useful.

Third, weather was assigned using a small number of weather stations. I think this is reasonable for a city-level project, but it may not capture very local weather variation across all NYC neighbourhoods.

Fourth, I did not include event calendars, concerts, sports games, subway disruptions, road closures, or flight arrival data. These could improve forecasts for event-heavy zones and airport zones, in my opinion.

Fifth, December test performance was weaker than November validation performance. This suggests that year-end demand patterns are harder to predict and may require more holiday-specific or event-specific features.

Sixth, the model uses lag features. I think that, these lag features must be updated continuously from recent actual demand.

Seventh, the final model predicts expected demand, but it does not directly solve dispatch optimisation. A separate optimisation layer would be needed to convert demand forecasts into taxi allocation decisions.

16. Final Recommendations

Based on this project, I recommend using the tuned LightGBM model as the final forecasting model because it produced the best validation WAPE and strong overall performance.

The forecast can support taxi allocation by identifying where and when demand is expected to be high. Fleet operators or dispatch planners could use the predictions to position taxis ahead of demand peaks, especially in high-demand Manhattan zones and airport zones.

For future improvement, I recommend:

1. Add more historical years of taxi data.
2. add event calendar data for concerts, sports, parades, and large public events.
3. Add flight arrival and departure data for JFK, LaGuardia, and Newark.
4. Add subway disruption or transit ridership features
5. Build a live feature pipeline that updates lag and rolling demand features hourly.
6. evaluate model performance separately by individual taxi zone, not only overall.
7. Build an optimisation layer that converts demand forecasts into recommended taxi allocation

17. Conclusion

I built an end-to-end machine learning pipeline for forecasting hourly yellow taxi demand across NYC taxi zones. I collected and prepared taxi, weather, and holiday data; engineered time, weather, holiday, lag, and rolling demand features; compared multiple regression models; selected LightGBM; improved it through hyperparameter tuning; evaluated it on a held-out test month; and saved a reusable model artifact for inference.

The final model forecasts the number of yellow taxi pickups per zone per hour. I hope this type of forecast can help support proactive taxi allocation, reduce passenger wait times, and improve the efficiency of urban mobility across New York City.